telcoin network

Telcoin Network node guide for mobile network operators

Deploy, sync, stake and operate a node

Version 1.4 · 15 October 2026

devs@telcoin.org

https://github.com/Telcoin-Association/tn-node-deployment

telcoin association

Contents

1 **Introduction** 4

1.1 What this guide covers 4

1.2 Contact 5

1.3 Vocabulary 5

1.4 Roles are decided on-chain 5

2 **Before you start** 7

2.1 Hardware per role 7

2.2 Operating system 7

2.3 Ports 8

2.4 Have these ready 9

2.5 Networks and endpoints 9

3 **Install** 10

3.1 Get the scripts 10

3.2 Run setup 10

3.3 Install methods 12

3.4 Where things land 13

3.5 Back up the keys now 13

3.6 Firewall 14

3.7 Optional node flags 14

4 **Confirm the node is syncing, then synced** 16

4.1 The epoch section 16

4.2 Checking by hand 17

4.3 How long it takes 17

5 **Day-2 operations** 18

5.1 Keep the scripts current 18

5.2 Update the node 18

5.3 One-way updates 19

5.4 Restarts and the epoch boundary 20

5.5 Change the configuration 21

5.6 Service commands 23

5.7 Node Manager UI 23

5.8 Testnet add-ons 24

5.9 Remove a node 24

6 **Public RPC (https and wss)** 25

6.1 DNS first 25

6.2 New install 25

6.3 Existing node ... 26
6.4 Verify ... 26
6.5 Turn it off ... 27

7 Validators: stake and activate ... 28
7.1 Prerequisites ... 28
7.2 Check the node with prepare-stake.sh ... 28
7.3 Send the stake ... 30
7.4 Activate ... 30
7.5 If prepare-stake.sh predicts a revert ... 31
7.6 Change the execution address before you stake ... 32
7.7 Watch your status ... 32
7.8 When you get a committee seat ... 33
7.9 Status values ... 34
7.10 Stake by hand ... 34
7.11 Rewards and exit ... 35
7.12 What not to do ... 36

8 Automation ... 37
8.1 Setup without prompts ... 37
8.2 Advertised RPC URL flags ... 38
8.3 Other scripts ... 39

9 Troubleshooting ... 40
9.1 Sync and node mode ... 41
9.2 Setup and preflight ... 42
9.3 Scripts and updates ... 43
9.4 Public RPC and certificates ... 44
9.5 Service and keys ... 45
9.6 Staking and exit ... 47

10 Getting help ... 48

1 Introduction

Telcoin Network is an EVM-compatible layer 1 blockchain. Its validators are GSMA-member mobile network operators, and the Telcoin Association governs the network. Consensus follows the Narwhal and Bullshark protocols, and the execution layer is reth, so every node serves Ethereum-style JSON-RPC. The public testnet is Adiri, chain ID 2017. The setup script lists mainnet as coming soon. Networks and endpoints (page 9) lists the chain IDs, public RPCs and explorers.

Anyone can run a node. You need no approval to install one, follow consensus and serve JSON-RPC. Validating is a separate, on-chain step. Only GSMA-approved mobile network operators can take it, and only after the Telcoin Association approves them.

The install is the same for everyone. Every node is provisioned validator-capable, and nothing in the install decides whether it validates. What separates a node that follows consensus from one that validates is on-chain state: a stake, an activation and a seat in the committee. The protocol reads that state each epoch and derives the node's role from it.

1.1 What this guide covers

This guide takes one Linux server from a fresh install to a synced Telcoin Network node and keeps it running. For operators the Telcoin Association has approved, it then walks through staking and activation as a validator.

Everything runs through the scripts in the deployment repository, https://github.com/Telcoin-Association/tn-node-deployment. You run them on the node itself. The validator steps add Foundry `cast` commands, which you run from the machine that holds your wallet. The repository's README is the reference for every script option. This guide is the path through them, in the order you need them:

- Before you start (page 7): hardware, operating system, ports, networks and what to have ready.
- Install (page 10): fetch the scripts, run setup, back up the keys, set up the firewall and choose the optional node flags.
- Confirm the node is syncing, then synced (page 16): how to tell a healthy node from a stuck one, how long sync takes, and what the health check says about epochs and the committee.
- Day-2 operations (page 18): script and node updates, one-way updates, restarts around the epoch boundary, configuration changes, service commands, the Node Manager UI, testnet add-ons and removal.
- Public RPC (https and wss) (page 25): serve JSON-RPC and WebSocket on your own DNS name.
- Validators: stake and activate (page 28): checking the node with `prepare-stake.sh`, staking, activation, committee seats, rewards and exit.
- Automation (page 37): run setup and the other scripts without prompts.
- Troubleshooting (page 40): symptoms, likely causes and fixes.
- Getting help (page 48): what to send when you ask for support.

The first five chapters apply to every operator. The validator chapter applies once the Association has approved you.

1.2 Contact

Email devs@telcoin.org with any question about running a node. The same address handles validator onboarding, approval and hardware. If you plan to validate, write before you buy hardware.

1.3 Vocabulary

Term	Meaning
Node	One `telcoin-network` install on one server, run by the `telcoin` systemd unit.
Epoch	The period between committee changes. Testnet epochs are six hours long.
Committee	The validators that vote on consensus during one epoch.
Validator	A node whose execution address has staked and been activated in the ConsensusRegistry contract. It votes only while it holds a committee seat.
Observer	Any node that is not in the current committee.
Execution address	The `0x` address you enter at key generation. It stakes, receives rewards, and is signed into the node's proof of possession.
BLS key	The node's consensus signing key, stored in `node-keys/` and encrypted with your BLS passphrase.
`node-info.yaml`	The node's public identity: BLS public key, proof of possession, P2P addresses and any advertised RPC URLs.
ConsensusNFT	A non-transferable token the Association mints to an approved execution address. Staking requires it.

1.4 Roles are decided on-chain

The protocol derives each node's role every epoch from committee membership. A node that is not in the current committee is an observer, whether or not it has staked. There is no setting in the install that makes a node a validator or an observer.

Observers come in two shapes:

- A follower keeps its RPC on `127.0.0.1` and serves only the box it runs on.
- A public-RPC observer also serves `https://` and `wss://` on its own DNS name through Caddy, and advertises those URLs in `node-info.yaml` so wallets and gateways can find it (see Public RPC (https and wss) (page 25)).

Becoming a validator takes four steps, in this order:

1. Association approval, which ends with a ConsensusNFT minted to your execution address (see Prerequisites (page 28)).
2. Stake (see Send the stake (page 30)).
3. Activation (see Activate (page 30)).
4. Committee selection (see When you get a committee seat (page 33)).

The node software does not change between them.

The node reports what it is doing through the `tn_nodeMode` JSON-RPC method:

`tn_nodeMode`	Meaning
`CvvActive`	Voting in the current committee.
`CvvInactive`	A committee member that is catching up before it can vote, for example after a long restart. This should be brief.
`Observer`	Not in the current committee. Every follower, every public-RPC observer, and every staked validator between seats reports this.

2 Before you start

2.1 Hardware per role

These figures come from the hardware requirements page in the telcoin-network documentation. CPU counts are physical cores. Cloud vCPUs are usually hyperthreads, so an 8 vCPU instance has about 4 physical cores.

Role	Minimum	Recommended
Observer, follower	2 cores, 8 GB RAM	4 cores, 16 GB RAM
Observer, public RPC	4 cores, 16 GB RAM	8 cores, 32 GB RAM
Validator	8 cores, 32 GB ECC RAM	16 cores, 64 GB ECC RAM

Storage is the same for every role: at least 2 TB of TLC NVMe (10,000 write IOPS, 300 MB/s), 4 TB recommended. Avoid QLC drives; they wear out under continuous chain writes.

Validators have two network requirements on top of that:

- 200 Mbps symmetric bandwidth at minimum, 1 Gbps recommended.
- A p95 round-trip time well below 1 second to at least 7 of the 10 committee members.

> **IMPORTANT**
>
> Email devs@telcoin.org with your planned hardware before you buy anything for a validator.

The same documentation page recommends turning swap off and running time sync (chrony or systemd-timesyncd) on every node. A validator holds its vote on a header timestamped ahead of its own clock until that lead has passed, on testnet today for a lead of up to one second. It rejects a header more than five to six seconds ahead, its drift tolerance plus the five-second vote timeout; a header between the two gets no vote yet, and its proposer retries. The deployment scripts check neither, so set both up yourself.

`setup-node.sh` runs a hardware check during preflight and prints one line per role, for example `[OK] Full node (follower): meets minimum` or `[WARN] Validator (minimum): below minimum`. The check only warns; it never blocks setup. It counts physical cores where the server reports them, so an 8 vCPU cloud instance with 4 physical cores shows as below the validator minimum. Its `Detected:` line says what it counted: `4 physical cores`, or `8 logical CPUs` plus a note when only the logical count can be read. It does not check for ECC memory or NVMe.

2.2 Operating system

`setup-node.sh` needs systemd 247 or newer, because it protects the BLS passphrase with systemd `LoadCredential`. That means Ubuntu 22.04 or later, Debian 12 or later, or RHEL 9 or later. Older systems stop with `systemd <version> detected -- version 247+ required.`

`setup-node.sh` does not support macOS.

A few parts assume a Debian-family system:

- Building from source installs its build dependencies with apt.
- `install-caddy.sh` (public RPC and the public dashboard) installs Caddy from its apt repository.
- `ui/install-ui.sh` installs Python with apt-get when Python 3 is missing.

Preflight also refuses to continue until the CVE-2026-31431 ("Copy Fail") kernel mitigation is in place. The check passes when a file under `/etc/modprobe.d/`, `/lib/modprobe.d/` or `/run/modprobe.d/` contains `install algif_aead /bin/false` and the `algif_aead` module is not loaded. Apply the mitigation from https://copy.fail before you start; you may need to unload the module or reboot. The README has the detection commands.

2.3 Ports

Port	Protocol	Open to	Purpose
49590	UDP	Everywhere, inbound	Primary P2P (QUIC). Every node.
49594	UDP	Everywhere, inbound	Worker P2P (QUIC). Every node.
443	TCP	Everywhere, inbound	Caddy, only when it serves public RPC or the dashboard.
80	TCP	Everywhere, inbound	Caddy certificate issuance and the http-to-https redirect. Recommended with Caddy.
8545	TCP	Loopback only	HTTP JSON-RPC. Never open it.
8546	TCP	Loopback only	WebSocket JSON-RPC. Never open it.
9101	TCP	Loopback only	Prometheus metrics.
43174	TCP	104.155.184.201/32, plus any monitoring hosts you add	Health-check endpoint, a testnet add-on. The node listens on it only when you enable the health monitor.
22 (or your SSH port)	TCP	Your admin IPs	SSH.

Open the two UDP ports on every node, including nodes that may never validate. A node that stakes later behind a closed port cannot reach the committee and misses consensus without any obvious error. The UDP ports you open must match the ports in the multiaddrs you advertise at key generation. 49590 and 49594 are conventions, not something the node enforces: it uses the ports in its own multiaddrs. If you set the node up on other ports, open those instead; `check-node.sh` and `firewall-setup.sh (1) View current firewall status` show the ports the node really uses, every worker included.

Open them in every layer that filters traffic: `ufw` on the host (see Firewall (page 14)), the cloud provider's firewall or security group, and the router's port forwards if the server is behind NAT.

2.4 Have these ready

- The execution address (`0x...`) the node will be registered under. For a validator this is the address that stakes and receives rewards, so it should live in a hardware wallet. It is signed into the proof of possession at key generation. Until it stakes, `prepare-stake.sh --rotate-address` can switch the node to another address (see Change the execution address before you stake (page 32)); after that it cannot change.
- A BLS passphrase, stored somewhere offline. The node needs it on every start.
- The server's public IP, and whether it sits behind NAT. Setup detects the public IP with api.ipify.org and the internal IP from the network interface, and builds the default multiaddrs from them. Correct them at the prompts if they are wrong.
- A mounted data drive with an `/etc/fstab` entry, if the chain data will not live on the boot disk.
- A DNS A record for the node's public name, if you want public RPC (see Public RPC (https and wss) (page 25)).
- Somewhere off the server to keep a backup of the node keys (see Back up the keys now (page 13)).
- For validators: an approval request with the Association in progress (devs@telcoin.org), and a machine with Foundry's `cast` and your wallet.

2.5 Networks and endpoints

Network	Chain ID	Public RPC	Explorer
Adiri testnet	2017 (`0x7e1`)	`https://rpc.adiri.tel`	`https://telscan.io` or `https://www.telscan.xyz`
Mainnet	487 (`0x1e7`)	`https://rpc.telcoin.network`, not launched yet	Not launched yet
Devnet	32285 (`0x7e1d`)	`https://rpc.devnet.telcoin.network`	None

The public RPCs sit behind a load balancer and answer JSON-RPC over HTTPS only; a WebSocket upgrade gets HTTP 405. For `wss://`, use a node that serves public RPC (see Public RPC (https and wss) (page 25)).

`https://rpc.telcoin.network` is reserved for mainnet, which has not launched, and today it answers for the testnet (chain 2017). Before you treat an endpoint as a given network, check its chain ID:

```
curl -s -X POST -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}' \
  https://rpc.telcoin.network
```

`"result":"0x7e1"` is the testnet, `0x1e7` mainnet and `0x7e1d` devnet. The scripts run this check themselves: when an RPC serves another chain than the node's, `check-node.sh` warns and skips its network comparison, and `prepare-stake.sh` stops.

Setup records the node's network in `/etc/telcoin/.node-meta` as `NETWORK=testnet`, `mainnet` or `devnet`. `check-node.sh`, `prepare-stake.sh` and the chain-config refresh in `edit-config.sh` take the network, and from it the public RPC, from that line.

3 Install

3.1 Get the scripts

Run the installer as your normal user, without `sudo`, so the scripts land in your own home directory at `~/telcoin-node-scripts`:

```
curl -fsSL https://install.telcoin.network | bash
```

If `install.telcoin.network` is unreachable, the raw GitHub URL is a drop-in replacement:

```
URL=https://raw.githubusercontent.com/Telcoin-Association/tn-node-deployment/main
curl -fsSL "$URL/install.sh" | bash
```

Or clone the repository yourself:

```
git clone https://github.com/Telcoin-Association/tn-node-deployment.git \
  ~/telcoin-node-scripts
```

If `~/telcoin-node-scripts` already exists, the installer asks `Re-install and overwrite? [y/N]`. To pick up newer scripts on an existing install, run `bash ~/telcoin-node-scripts/update-scripts.sh` instead (see Keep the scripts current (page 18)).

3.2 Run setup

```
sudo bash ~/telcoin-node-scripts/setup-node.sh
```

Setup is interactive and runs in the order below. Most prompts show their default in brackets; press Enter to accept it. Yes/no prompts default to No.

1. Welcome. Answer `y` to `Ready to begin node setup?`.
2. Preflight ("Step 1 of 8"). Setup checks that it runs as root, detects the distribution and runs the CVE-2026-31431 gate. It lists your mounted filesystems and physical disks, then asks `Data directory [/var/lib/telcoin]`. Enter a path on your data drive if you have one. It then runs the hardware check, an internet check and a port check (in-use ports only warn), installs `curl` and `git` if missing, and checks for systemd 247 or newer.
3. Network. Choose `1) Adiri Testnet (Chain ID: 2017).` `2) Mainnet (Chain ID: 487)` is marked coming soon and returns you to the menu. Devnet is not offered interactively.
4. Testnet add-ons (testnet only). Each is off by default: the health-monitor endpoint on TCP 43174, log shipping, metrics shipping, a region label if you turned either shipping option on, and VPN admin SSH. Read the testnet add-ons guide before you opt in.
5. Install method (see Install methods (page 12)). Source builds first ask `Install missing packages now?` if build dependencies are missing (answer y; No exits), then show a version picker whose default on testnet is the latest `-adiri` release tag. On testnet a release older than `v0.13.0-adiri` stops setup

before checkout; `main` or a commit only warns. Docker installs ask for the image, defaulting to the newest `-adiri` tag, and hold a typed image to the same floor. "I already have it" first offers a binary it finds (`Use this binary?`) and asks `Full path to telcoin binary:` only if it finds none or you decline.

6. BLS passphrase protection (not asked for Docker). This menu comes after the source build (20 to 40 minutes), or for an existing binary once setup has found it. For source builds the passphrase question comes only after the build finishes, so stay attached to the session. `1) systemd LoadCredential` suits most operators. `2) TPM/vTPM sealing` ties the passphrase to this machine's TPM; it falls back to LoadCredential if no TPM is found. Type `1` or `2`; Enter alone is not accepted.
7. Node configuration ("Step 3 of 8"). First, RPC access: `1) Private` keeps RPC on `127.0.0.1`; `2) Public` asks for the node's DNS name, then always asks for an optional inbound public IP (Enter auto-detects; set it behind NAT; see Public RPC (https and wss) (page 25)). Then the ports: P2P primary `49590`, P2P worker `49594`, RPC `8545`, metrics `9101`. Then `Use these default paths?` for the config, log and install directories, and an optional advertised node name.
8. System infrastructure ("Step 4 of 8"). Service user and group, both `telcoin` by default. The user gets no login shell. Docker installs give it UID 1101. Docker and existing installs may be asked `Clone the repository now to get the chain-config files?`. Answer y, because No means copying the chain configs by hand before step 10. If you passed node flags (see Optional node flags (page 14)), setup asks the installed release here whether it has them, and stops before any key is made if it does not.
9. Keys ("Step 5 of 8"). If `node-keys/` already exists, setup asks `Overwrite existing keys?`. On a re-run, answer No (see What not to do (page 36)). Otherwise it asks for the execution address and prints the detected public IP (it asks for one only if detection fails). It then asks for the external primary and worker addresses and, after printing the detected internal IP, the listener primary and worker addresses. Each shows a default built from the detected IP, such as `/ip4/<public-ip>/udp/49590/quic-v1`; check the IP in them. Then it asks for the BLS passphrase twice. The keytool writes the keys and `node-info.yaml`, setup prints `node-info.yaml` and the staking steps (with the live stake amount and a pointer to `prepare-stake.sh`), and it waits at `Press Enter to confirm you have backed up your keys`. Do the backup in Back up the keys now (page 13) before you press Enter.
10. Configuration ("Step 6 of 8"). Copies `genesis.yaml`, `committee.yaml` and `parameters.yaml` into the data directory. If it cannot find them, it prints where to get them and waits for you to copy them in.
11. Service ("Step 7 of 8"). Writes the start wrapper, the systemd unit and `/etc/telcoin/.node-meta`, then asks `Start the node now?`. Only if you answer yes and the service comes up does it ask `Enable auto-start on server reboot?`. Answer yes to both.
12. Public RPC, only if you gave a domain and the node started. Setup checks DNS and enables the endpoint. If DNS is not ready yet, setup still finishes and prints the exact command to run later.
13. The testnet add-ons you opted into are applied. Log or metrics shipping asks for your ingest token (hidden; leave it blank to skip).
14. A summary with the paths, the P2P ports `node-info.yaml` advertises (every worker), a firewall reminder for those ports, the next steps and a link to the operator runbook.

`sudo bash ~/telcoin-node-scripts/setup-node.sh --help` lists the public RPC flags and the node flags. An argument setup does not recognise is reported on stderr as `Unknown argument: <arg>`, and setup carries on without it, so check the spelling of any flag you pass. Public RPC (https and wss) (page 25) covers the public RPC flags, Optional node flags (page 14) the node flags, and Automation (page 37) the flags for unattended runs.

3.3 Install methods

Menu choice	`--install-method`	What it does	How it updates
`1) Build from source`	`source`	Installs Rust and the build dependencies, clones telcoin-network to `/opt/telcoin-source`, builds it (20 to 40 minutes; log in `/tmp/tn-build.log`) and installs `/opt/telcoin/telcoin-network`.	`update-node.sh`
`3) Docker`	`docker`	Installs Docker if missing and pulls the image from `us-docker.pkg.dev/telcoin-network/tn-public/adiri`. The default tag is the newest `-adiri` tag in the registry, or `v0.16.0-adiri` if the registry cannot be reached.	`update-node.sh`
`4) I already have it`	`existing`	Uses a `telcoin-network` binary you supply.	By hand (see Update the node (page 18)).
`2) Pre-built binary`	none	Not available yet; the menu sends you back.	

The README has more on install options and network binding.

3.4 Where things land

Path	What it is
`/opt/telcoin/telcoin-network`	The node binary for source builds. An `existing` install runs the binary from the path you gave setup (`Binary=` in its summary).
`/opt/telcoin/start-telcoin.sh`	Start wrapper. It reads the BLS passphrase and holds the node's launch command and flags.
`/etc/systemd/system/telcoin.service`	The systemd unit, `telcoin`. Docker installs also name the container `telcoin`.
`/var/lib/telcoin/node-keys/`	BLS and P2P keys. Back these up.
`/var/lib/telcoin/node-info.yaml`	Public identity and advertised URLs.
`/var/lib/telcoin/genesis/`, `/var/lib/telcoin/parameters.yaml`	Chain configuration.
`/var/lib/telcoin/db/`	Chain database. This is what grows.
`/var/lib/telcoin/consensus-db/state_exports/`	One directory per epoch when state export is on (see Optional node flags (page 14)).
`/etc/telcoin/bls-passphrase`	BLS passphrase, mode 600, owned by the service user.
`/etc/telcoin/bootstrap-peers.yaml`	The peers map, when the node runs with `--bootstrap-peers` (see Optional node flags (page 14)). World-readable.
`/etc/telcoin/.node-meta`	Install settings the other scripts read, one `KEY=value` per line, among them `NETWORK`, `INSTALL_METHOD`, `DOCKER_IMAGE` or `BINARY_PATH`, `DATA_DIR`, `RPC_PORT`, `VALIDATOR_ADDRESS`, `BOOTSTRAP_PEERS_FILE`, `STATE_EXPORT`, and `PUBLIC_RPC_DOMAIN`, `PUBLIC_RPC_URL` and `PUBLIC_WS_URL` while public RPC is on. Root-only (mode 600), so run the scripts with `sudo`.
`/var/log/telcoin/telcoin.log`, `telcoin-error.log`	Node output.
`/opt/telcoin-source/`	The telcoin-network source tree (source builds).

If you chose a different data directory, read it in place of `/var/lib/telcoin` everywhere in this guide. The README's system layout shows the full tree.

3.5 Back up the keys now

> **WARNING**
>
> Losing `node-keys/` means losing the node's identity. If you have not staked yet, you can regenerate keys and carry on. Once you have staked, you have to go back to the Association.

Back up `node-keys/` and `node-info.yaml` together. Keep the BLS passphrase somewhere else (a password manager, or paper in a safe), never next to the archive. In a second terminal on the server:

```
(umask 077; sudo tar -C /var/lib/telcoin -czf - node-keys node-info.yaml \
  > ~/telcoin-keys.tgz)
```

Copy the archive off the server from your own machine, then delete it from the server:

```
scp user@SERVER_IP:telcoin-keys.tgz .
ssh user@SERVER_IP 'shred -u ~/telcoin-keys.tgz'
```

Treat the archive as a secret. Then go back to setup and press Enter.

3.6 Firewall

```
sudo bash ~/telcoin-node-scripts/firewall-setup.sh
```

Choose `2) Enable firewall with recommended defaults`. It sets inbound to deny and outbound to allow, allows SSH, allows TCP 43174 from the Association monitor only, and allows the node's P2P ports when a node is installed. It reads those ports from the node's start wrapper and `node-info.yaml`, every worker included, so a node set up on other ports than 49590 and 49594 gets the right rules. When Caddy serves the public RPC or the dashboard on this server, it also allows TCP 80 and 443 before it turns `ufw` on, so the site stays up. The plan it shows before you confirm lists every rule. It asks whether to reset to a clean slate first; No keeps your existing rules.

SSH stays open to the whole internet after this. To restrict it, open `5) Manage trusted IP whitelist` and add your own IP or range first with `1) Add trusted IP for SSH access`. Adding a whitelist entry does not remove the blanket rules. Then remove both blanket SSH rules, the ones for your SSH port with `Anywhere` as the source: the IPv4 rule and its `(v6)` twin, which `ufw` adds alongside it. `2) Remove a whitelist entry` deletes one rule number per run, and the numbers shift after a delete, so read the numbered list it prints again before you remove the second rule. Test SSH from a new terminal before you close the current one.

`ufw` is only the host layer. Open the same UDP ports in the cloud firewall or security group as well, and forward them on the router if the server sits behind NAT. `1) View current firewall status` lists each P2P port with its label, for example `UDP 49594 is open (worker)`. The README covers the rest of the firewall script.

3.7 Optional node flags

Setup can pass two extra features to the node. Both are optional, and `edit-config.sh` can add, change or remove them after the install (see Change the configuration (page 21)).

`--bootstrap-peers <file>` replaces the bootstrap servers in the genesis with your own list of peers to dial at start. It needs `v0.15.0-adiri` or later. The node treats these peers as discovery hints only: it does not keep them connected ahead of other peers or exempt them from load penalties.

- The file is a YAML or JSON map keyed by BLS public key. Each value is that node's `primary` and `workers` entries, copied from under `p2p_info` in its `node-info.yaml`. The README's bootstrap peers section shows the shape.
- It must be a regular file of at most 64 KiB that other users can read (`chmod 644`). The map does not stay private: setup installs it world-readable as `/etc/telcoin/bootstrap-peers.yaml`, and its check passes the map on the node binary's command line. A file its group or other users cannot read is refused, with the `chmod` to run.
- Setup checks the map with the release's own parser and stops with the parser's reason if a key or value is wrong.
- The start wrapper passes `--bootstrap-peers "$(cat /etc/telcoin/bootstrap-peers.yaml)"`, so the node reads the file again at every start. An edit to that file takes effect at the next restart without any check, and if the file goes missing or empty the node does not start. `edit-config.sh` checks a new map before it installs it.

`--enable-state-export` (`v0.13.0-adiri` or later) writes each epoch's final execution state under `<data dir>/consensus-db/state_exports/epoch-N/`. Every export is a full copy of the state and none is deleted, so the disk fills over time. `--state-export-keep <n>` keeps only the newest n exports (1 to 999999) and turns export on by itself; it needs `v0.15.0-adiri` or later. A value of 2 or more keeps the previous export while the next one is written.

```
sudo bash ~/telcoin-node-scripts/setup-node.sh \
  --bootstrap-peers ./peers.yaml --state-export-keep 3
```

Setup asks the release it installs whether it has each flag you give, during System infrastructure ("Step 4 of 8"), and stops before any key is made when it does not. It records the choices in `.node-meta` as `BOOTSTRAP_PEERS_FILE` (the installed path, empty when none) and `STATE_EXPORT` (`off`, `unlimited` or a number).

The node also reads `network-config`, a YAML file in the data directory; the scripts and the Node Manager UI write only its `hostname` line, the node name it advertises. Four of the keys it accepts are opt-in, `process_budget`, `source_admission`, `trusted_nodes` and `committee_peers`: they are unset by default (absent, `null` or `{}`), and no script sets them. Leave them unset unless devs@telcoin.org asks you to set one. The `source_admission` key can refuse committee peers once its limits fill, and a node that has it set logs a warning at start that begins `source admission is enabled`.

4 Confirm the node is syncing, then synced

Check that the service is up and watch its log for a minute:

```
systemctl status telcoin
journalctl -u telcoin -f
```

Then run the health check:

```
sudo bash ~/telcoin-node-scripts/check-node.sh
```

It compares your node with the public RPC of the network in `.node-meta` (`https://rpc.adiri.tel` on testnet; see Networks and endpoints (page 9)), or with the RPC you pass as `--network-rpc <URL>`, and ends with one verdict:

- `CATCHING UP: <N> execution blocks behind` while the node is more than 50 blocks behind the network.
- `All checks passed -- node is healthy and caught up (EVM lag <N>)` once it has caught up.
- `<N> issue(s) found -- review warnings/errors above` when something else is wrong.

Run it twice a few minutes apart. The second run also reports how far the local block advanced since the first, which tells you whether the gap is closing. The report header has a `Network:` line with the network, its chain ID and where that came from. The first section lists what the node runs: the Docker image or binary, the bootstrap peers and state export when set, the P2P ports and the RPC URLs it advertises. When the node binary supports `tn_nodeMode`, the report also prints a `Consensus role:` line. The script always exits 0, so anything that consumes it should read the verdict line, not the exit code.

If the comparison RPC serves another chain than the node's, the report says so and skips the comparison instead of blaming the node, for example `https://rpc.telcoin.network serves chain 2017 (testnet), not chain 487 (mainnet) -- skipping the network comparison`. It does not count as an issue; `--network-rpc <URL>` with an RPC for the node's chain brings the comparison back.

4.1 The epoch section

The `Checking epoch and committee...` section shows the current epoch, the time to the next boundary and its UTC time, and committee membership for this epoch and the next two, for example `epoch 581: not a member · 582: not a member · 583: member`. "Not a member" is normal for a node that is not a validator, for a validator before its first seat, and for an Active validator between seats. Such a node also gets the info line `Not in recent headers -- expected: this node is not in the committee for epoch 581`. Only a committee member missing from the latest headers is an error.

For a staked validator the section adds the activation epoch and the earliest committee seat (see When you get a committee seat (page 33)). It also compares the node's workers with the number the network requires (`WorkerConfigs.numWorkers()`, 1 on testnet today); fewer is an error.

4.2 Checking by hand

Do not use `eth_syncing`. On Telcoin Network it returns `false` even while the node is catching up. Compare `eth_blockNumber` on your node with the public RPC instead:

```
bn() { curl -s -X POST -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_blockNumber","params":[]}' "$1" \
  | sed -E 's/.*"result":"(0x[0-9a-fA-F]+)".*/\1/'; }
echo "local:   $(( $(bn http://127.0.0.1:8545) ))"
echo "network: $(( $(bn https://rpc.adiri.tel) ))"
```

The node is synced when the two numbers stay within a few blocks of each other.

To see the node's consensus role:

```
curl -s -X POST -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"tn_nodeMode","params":[]}' \
  http://127.0.0.1:8545
```

Any node outside the committee returns `"Observer"`, including a staked validator that has not been seated yet.

4.3 How long it takes

There is no published figure for a first sync; it depends on chain height, disk and bandwidth. Watch the lag shrink rather than waiting for a fixed time. Each restart replays recent consensus output before the node catches up again, so a node that is restarted often stays behind. If the lag does not shrink between runs, see Troubleshooting (page 40).

5 Day-2 operations

5.1 Keep the scripts current

```
bash ~/telcoin-node-scripts/update-scripts.sh
```

Run it as your normal user, without `sudo`. It compares every tracked file with the latest version on GitHub, asks `Download and install all updates? [Y/n]`, and checks each download against its published SHA-256 before installing it. A file whose checksum is missing or does not match is not installed; the run lists it and exits 1, so run it again later. New scripts, such as `prepare-stake.sh`, arrive the same way. If the Node Manager UI is installed and has changed, it redeploys the UI too and may ask for your sudo password.

Run it before every node update. `update-node.sh` learns how to handle new releases through these updates.

5.2 Update the node

```
sudo bash ~/telcoin-node-scripts/update-node.sh
```

The script works out whether the node was built from source or runs in Docker and offers the matching versions. Source installs get the same version picker as setup, defaulting to the latest `-adiri` tag. Docker installs get the recent registry tags, newest first, with the latest and current ones marked. Then it asks how far to go:

- `1) Prepare only` builds the binary or pulls the image while the node keeps running. Apply it later in a quiet window.
- `2) Prepare AND apply` also stops the node, swaps in the new version, restarts it and checks its health.
- `3) Cancel`, or any other input, cancels.

A prepared update survives between runs. The next run offers `1) Apply now`, `2) Discard and prepare a different update` or `3) Exit`. `sudo bash ~/telcoin-node-scripts/update-node.sh --discard` drops a prepared update after asking `Discard this pending update?`.

What else to expect:

- On a node registered on-chain as a validator (status 1 to 4), or one whose status cannot be read, the script shows a downtime warning and asks you to type `CONFIRM` before it stops the node.
- When the node votes in the current committee and the epoch boundary is close, the apply waits before it stops the node (see Restarts and the epoch boundary (page 20)). `--no-epoch-wait` skips the wait.
- After the restart it waits up to 45 seconds (600 on a one-way update) for the service to be active and for the node to answer `tn_latestConsensusHeader`. When many nodes restart at once, quorum takes longer to form. Raise the window with `TN_UPDATE_VERIFY_TIMEOUT`, in seconds, for example `sudo TN_UPDATE_VERIFY_TIMEOUT=120 bash ~/telcoin-node-scripts/update-node.sh`. A number you set always wins, on a one-way update too, so leave it unset there or give it 600 or more.
- If that check fails, it offers to roll back to the previous binary or image, except after a one-way update (next item).

- When the node runs a release older than `v0.16.0-adiri` and the target is `v0.16.0-adiri` or newer (or a branch, commit or digest), the update is one-way. The new release migrates the consensus store on its first start, and older releases cannot open the data directory afterwards. Snapshot the data directory before you apply (see One-way updates (page 19)). The script says the update is one-way, warns when the free space under `consensus-db/epochs` is less than twice its largest `epoch-N` directory, asks whether you have a snapshot, and waits up to 600 seconds. If the check fails it does not roll back and does not stop the node, which may still be migrating. When the script cannot read which release the node runs, it treats the update as one-way.
- Only one update runs at a time. A second run stops with `Another update is already running (PID N).`
- It never touches the keys, `node-info.yaml`, the BLS passphrase, the chain configuration, or the listener addresses.
- A ref you type at the version picker cannot start with `-`; the script refuses it with `Invalid ref "<ref>": a ref cannot start with "-".` before any git command runs.

The script does not update `existing` installs. For one, it names the binary the node runs (the one its start wrapper launches, recorded as `BINARY_PATH` in `.node-meta`) and the steps to replace it: stop the node, install the new release binary over that file, and start the node again.

```
sudo systemctl stop telcoin
sudo install -m 0755 <new binary> <binary path>
sudo systemctl start telcoin
```

On a committee node, run only the `install` line, which replaces the file while the node runs, then restart with `edit-config.sh` menu item `12) Restart node`, which waits for the epoch boundary. After any update, run `sudo bash ~/telcoin-node-scripts/check-node.sh`.

5.3 One-way updates

An update from a release older than `v0.16.0-adiri` to `v0.16.0-adiri` or newer cannot be undone by putting the old binary or image back, because the new release migrates the consensus store on its first start. Before such an update, snapshot the data directory with the node stopped. Prepare first (`1) Prepare only`), so the node is down only for the copy:

```
sudo systemctl stop telcoin
sudo cp -a /var/lib/telcoin /var/lib/telcoin.pre-v0.16   # or take a disk snapshot instead
sudo systemctl start telcoin
```

The copy needs as much free space as the data directory; a disk snapshot from your cloud provider needs none on the server. On a committee node, stop it well away from an epoch boundary; `check-node.sh` shows the time to the next one. Then run `update-node.sh` again, choose `1) Apply now`, and answer yes when it asks about the snapshot. The first start can take minutes while the node migrates each epoch it opens; `journalctl -u telcoin -f` shows the progress. If the health check fails, the script leaves the node on the new release and does not stop it. A node that is still migrating comes up without help, and a node that stopped says why in the journal, so read the journal before you decide to go back. A node whose epoch database fails its start-up check stops with `Cannot check database (epoch)` in the journal; the fix is in Service and keys (page 45). That fix touches only `consensus-db/epoch`, which holds the epoch in progress,

and never `consensus-db/epochs`, the epoch packs this update migrates. To go back, stop the node, move the migrated data directory aside and the snapshot back into its place (for the copy above, `/var/lib/telcoin.pre-v0.16` back to `/var/lib/telcoin`), run the copy command from the error message to put the old binary or launch file back, and start the node. An older release cannot start on the migrated data directory; it stops with `invalid version (should be 0)`.

5.4 Restarts and the epoch boundary

A node that votes in the committee should not be down when the epoch changes: the epoch closes at the first commit at or after the boundary, and the committee exchanges epoch votes for 60 to 75 seconds afterwards. So these scripts check before they restart the node: `update-node.sh` when it applies an update, `edit-config.sh`, `install-caddy.sh` (`rpc-enable` and `rpc-disable`), `setup-observability.sh` and `prepare-stake.sh --rotate-address`. Other restarts happen at once: `systemctl`, and in the Node Manager UI the Start, Stop and Restart buttons, the tracing toggle and saving the advertised node name.

When the node reports `CvvActive` and the boundary is five minutes away or less, the script waits until the epoch closes, then 90 seconds more for the new committee to settle, never longer than 30 minutes in all. It prints a line that starts `Waiting for epoch <N> to close before restarting`, with the time to the boundary and the limit, then a progress line every 15 seconds. Nodes outside the committee, staked validators between seats included, are not delayed, and a restart that rolls back a failed change never waits.

To restart at once, pass `--no-epoch-wait` (`update-node.sh` and `edit-config.sh`) or set `TN_SKIP_EPOCH_WAIT=1` through `sudo`, for example `sudo TN_SKIP_EPOCH_WAIT=1 bash ~/telcoin-node-scripts/update-node.sh`. On a committee node, check the time to the next boundary with `check-node.sh` before a restart that does not wait (see The epoch section (page 16)), or restart with `edit-config.sh` menu item `12) Restart node`, which waits.

If a committee node is stopped at the epoch boundary anyway, it repairs its epoch records when it starts again and logs warnings that need no action (see Service and keys (page 45)). It still misses the epoch votes while it is down, so keep to the wait.

These environment variables tune the wait:

Variable	Default	Meaning
`TN_SKIP_EPOCH_WAIT`	unset	`1` skips the wait.
`TN_EPOCH_MARGIN`	300	Seconds before the boundary inside which a restart waits.
`TN_EPOCH_SETTLE`	90	Seconds to wait after the epoch changes.
`TN_EPOCH_WAIT_MAX`	1800	Cap on the whole wait, settling included. `0` turns the wait off.
`TN_EPOCH_POLL`	15	Seconds between progress lines, held between 5 and 20.

The Node Manager UI runs the same scripts, so an update or a config save there can wait too (see Node Manager UI (page 23)).

5.5 Change the configuration

```
sudo bash ~/telcoin-node-scripts/edit-config.sh
```

Menu item	What it changes
`1) Listener addresses`	`PRIMARY_LISTENER_MULTIADDR` and `WORKER_LISTENER_MULTIADDR`.
`2) Metrics address`	The `--metrics` address; `off` removes the flag.
`3) Log verbosity`	`-v` to `-vvvvv`.
`4) RPC access`	Not for public RPC (see the cautions below).
`5) BLS passphrase`	The passphrase file. It never re-encrypts the keys.
`6) P2P ports`	The P2P listener ports.
`7) Docker image`	The image a Docker install runs.
`8) Bootstrap peers`	A peers map in place of the genesis list (see Optional node flags (page 14)), or none.
`9) State export`	Off, every epoch's export, or the last N epochs.
`10) Private forward targets`	`allow_private_forward_targets` in `parameters.yaml`; private networks only.
`11) Refresh chain configs`	Pulls the latest genesis, committee and parameters.
`12) Restart node`	
`13) Exit`	

Before the first change to a file, it copies the file to `<file>.bak.<UTC time>`. After an edit it asks `Restart the node now to apply changes?`; No keeps the change for the next restart.

For a single edit without the menu, use `--set`, with or without `--json`:

```
sudo bash ~/telcoin-node-scripts/edit-config.sh --set state_export=3
```

`--set` restarts the node and checks that it stays up; if it does not, it puts back every file the edit changed and restarts the node on the old configuration. A run stopped before the restart, during the epoch wait for example, puts the files back too. When the value is already in place, nothing is written and the node is not restarted.

Field	Values
`primary_listener`, `worker_listener`	A multiaddr: `/ip4/<addr>/udp/<port>/quic-v1` or `/ip6/...`.
`metrics`	`<IPv4>:<port>` adds or changes `--metrics`; `off` removes it.
`verbosity`	`-v`, `-vv`, `-vvv`, `-vvvv` or `-vvvvv`.
`docker_image`	`<registry/path:tag>`, Docker installs only. The image is pulled first.
`bootstrap_peers`	An absolute path to a peers map, checked as at setup and installed as `/etc/telcoin/bootstrap-peers.yaml`, or `none` to remove the flag and the file. Needs `v0.15.0-adiri` or later.
`state_export`	`off`, `unlimited` (every export kept; `v0.13.0-adiri` or later) or a number of epochs to keep, 1 to 999999 (`v0.15.0-adiri` or later).
`allow_private_forward_targets`	`true` or `false`. `true` lets the node forward transactions to committee RPC endpoints on private addresses. `edit-config.sh` refuses `true` on testnet and mainnet, so it is for private networks such as a devnet.

Each node flag is checked against the installed release before anything changes. Every edit takes the update lock, so it is refused while `update-node.sh` applies an update: `an update is in progress (PID N); try again when it has finished`. Before it restarts a node that votes in the current committee, `edit-config.sh` waits for the epoch boundary (see Restarts and the epoch boundary (page 20)); `--no-epoch-wait` skips the wait.

`11) Refresh chain configs` takes the network from `.node-meta` and refuses unless the node's genesis and the new one declare the same chain ID. On a source build checked out at a release tag, `git pull` fails and the refresh stops with `git pull failed in /opt/telcoin-source. Chain configs unchanged.`

Two cautions:

- For public RPC, use `install-caddy.sh` (see Public RPC (https and wss) (page 25)), not the menu's `4) RPC access` item. reth must stay on `127.0.0.1`.
- Change the P2P ports only to fix a conflict. The new ports must be open in every firewall layer and must match the multiaddrs in `node-info.yaml`.

The README covers editing the configuration in more detail.

5.6 Service commands

```
sudo systemctl start telcoin
sudo systemctl stop telcoin
sudo systemctl restart telcoin
journalctl -u telcoin -f
sudo tail -f /var/log/telcoin/telcoin.log
sudo systemctl reset-failed telcoin   # after too many failed restarts
```

A restart with `systemctl` does not wait for the epoch boundary (see Restarts and the epoch boundary (page 20)). The README has the full command reference.

5.7 Node Manager UI

The optional web UI shows health, logs and configuration, and can run updates. Install it on the node:

```
sudo bash ~/telcoin-node-scripts/ui/install-ui.sh
```

The installer validates the sudo whitelist for the UI's privileged helper with `visudo` before it changes anything that is running. If Flask cannot be installed, a UI source file is missing, the helper does not parse or `visudo` rejects the whitelist, it stops there and the running UI stays as it was. It installs the whitelist last; if a later step fails, run the installer again.

The UI listens on `127.0.0.1:8080` only. Reach it from your own machine through an SSH tunnel. Use either the helper script from a local copy of the deployment repository, or a tunnel by hand:

```
./open-ui.sh user@SERVER_IP
ssh -L 8080:localhost:8080 user@SERVER_IP   # then open http://localhost:8080
```

To reach the dashboard on a public HTTPS name, run `sudo bash ~/telcoin-node-scripts/install-caddy.sh` and choose `[1] Dashboard`. Caddy then serves it behind a login. Public access is read-only; management stays on the tunnel. The dashboard needs its own hostname, such as `dashboard.<your-node-domain>`, separate from the public RPC name. Both names must be DNS names with two or more labels, not IP addresses. See the README's web UI and external dashboard access sections.

`update-scripts.sh` redeploys the UI when it changes. If the UI then shows `The privileged helper is outdated.`, the helper is older than the page (this release needs helper API 2), and updates, config saves and public access fail until you re-run `sudo bash ~/telcoin-node-scripts/ui/install-ui.sh`.

What to expect from the UI:

- The setup wizard can make the node public at install time. In the Configure step, choose the `Public` card under RPC access and give the node's hostname; the field suggests the node's own name, and `Check DNS` tests it. A DNS mismatch is only a warning, because setup checks DNS again and keeps RPC private until the name resolves (see New install (page 25)). The review step reads `Public at <hostname>` or `Private`, and the completion card then shows whether public RPC is on and what `node-info.yaml` advertises.

- The validator view follows the chain. The UI asks the network's public RPC for the node's `getValidator` record, then the synced node, then its last saved answer, so a newly staked validator gets the validator view while it is still syncing. A banner says when the view comes from a cached answer, or when the role is unknown and the view defaults to observer.
- The Config tab has no fields for bootstrap peers, state export or private forward targets yet; set those with `edit-config.sh` (see Change the configuration (page 21)).
- Updates and config saves run the same scripts, so they wait for the epoch boundary too (see Restarts and the epoch boundary (page 20)). A progress pane can show `Waiting for epoch N to close` for up to 30 minutes, with a line every 15 seconds. Keep the tab open until the result line appears, because leaving the tab stops the script. Leaving an update during the wait cancels it before the node is stopped, and the prepared update stays for a later apply; leaving it once the apply has started can cut the apply off part way. A config save that is stopped before it restarts the node is rolled back: every file it changed is put back as it was.
- A one-way update (see One-way updates (page 19)) shows its warning in the progress pane when you prepare it. Apply does not ask about a snapshot, so take it before you press Apply, and expect the health check to take up to 10 minutes after the restart.

5.8 Testnet add-ons

You can turn on the health monitor, log shipping, metrics shipping and VPN admin SSH after setup:

```
sudo bash ~/telcoin-node-scripts/setup-observability.sh   # health, logs, metrics
sudo bash ~/telcoin-node-scripts/setup-vpn.sh             # VPN admin SSH
```

Turning on the health monitor, log shipping or metrics shipping adds flags to the node command (`--healthcheck`, `--log.file.format json`, `--metrics`) and restarts the node. On a committee node that restart waits for the epoch boundary (see Restarts and the epoch boundary (page 20)). If a flag cannot be added, for example because the node command ends in a `#` comment, the enable fails without restarting the node, names the flags to add by hand, and does not record the add-on as on. Add the flags and run it again.

Read the testnet add-ons guide for what each one shares and how to turn it off. Read the WireGuard VPN guide before you enrol in the VPN.

5.9 Remove a node

If the node serves public RPC, turn that off first, because `remove-node.sh` does not touch Caddy:

```
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-disable
sudo bash ~/telcoin-node-scripts/remove-node.sh
```

The removal script asks before each component: service, Docker container and image, chain data, keys, binary, source tree, logs, service user, and the UI. Deleting the keys needs you to type `DELETE`. Do not delete the keys of a staked validator unless you hold a tested backup. Menu option `2) Wipe chain data only` keeps the keys and configuration and makes the node resync from scratch.

6 Public RPC (https and wss)

Public RPC serves your node's JSON-RPC at `https://<domain>/` and its WebSocket at `wss://<domain>/`. It also advertises both URLs in `node-info.yaml` so wallets and gateways can discover the node. reth itself stays on `127.0.0.1`. Caddy holds the TLS certificate on port 443 and proxies to reth on loopback.

Turning it on is a deliberate choice. The Caddy site has no authentication and no rate limiting, so anyone can send requests to it. It suits observers that are meant to serve the public.

> **IMPORTANT**
> Other nodes forward transactions to the RPC URLs the committee advertises. A validator therefore has to advertise a reachable RPC URL to receive forwarded transactions. Choose that URL deliberately and protect the endpoint behind it. The Caddy site has no rate limiting of its own, so put rate limits in front of it, keep reth on `127.0.0.1` behind Caddy, and allow nothing else through the firewall.

The README section on the public RPC endpoint covers the internals. Its public node tuning section lists the reth request limits and caches to review before you serve the public. The steps follow.

6.1 DNS first

Create an A record for the node's public name that points at the server's inbound public IP. Wait until it resolves before you enable anything. Caddy requests a Let's Encrypt certificate as soon as the site loads. If the name does not point at the server yet, issuance fails and repeated attempts get rate-limited.

- Allow inbound TCP 443 (required) and TCP 80 (recommended) in every firewall layer, and forward both on the router if the server is behind NAT.
- Behind NAT, or on a server with several addresses, pass the inbound IP with `--public-ip <ip>`. The address the scripts detect is the outbound one, which can differ.
- The RPC name and the dashboard name must be different, for example `node7.example.com` for RPC and `dashboard.node7.example.com` for the dashboard.
- Caddy 2.8.0 or newer. `install-caddy.sh` installs Caddy from its apt repository when it is missing, and refuses an older one before it writes anything, with the install instructions.

6.2 New install

```
DOMAIN=node7.example.com   # your node's public name
sudo bash ~/telcoin-node-scripts/setup-node.sh --rpc-domain "$DOMAIN"
```

Behind NAT, run this instead, with the inbound public IP:

```
sudo bash ~/telcoin-node-scripts/setup-node.sh --rpc-domain "$DOMAIN" \
  --public-ip 203.0.113.10
```

Or answer `2) Public` at the RPC access prompt. With a domain, key generation writes `https://<domain>/` and `wss://<domain>/` into `node-info.yaml` when the release's keytool supports advertising RPC (`--rpc-http`, present since `v0.12.0-adiri`). After the node starts, setup checks DNS and runs `install-caddy.sh --phase=rpc-enable`. That run does not restart the node when `node-info.yaml` already holds those URLs. If DNS is not ready, setup finishes anyway and prints the command to run later.

`--no-public-rpc` keeps RPC private without asking. It cannot be combined with `--rpc-domain` or `--rpc-public`; setup stops with an error (an `error` event in `--json` mode). `--rpc-public` without `--rpc-domain` leaves RPC private and prints a warning with the command to enable it later.

6.3 Existing node

Check DNS first, then enable:

```
DOMAIN=node7.example.com   # your node's public name
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-check-dns \
  --rpc-domain "$DOMAIN"
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-enable \
  --rpc-domain "$DOMAIN"
```

`rpc-check-dns` changes nothing and exits 0 only when the name reaches this server. `rpc-enable` writes the Caddy site and reloads Caddy, and adds `ufw` rules for 80 and 443, also while `ufw` is off, so they are in place when you turn it on. It makes sure reth serves WebSocket, and sets the advertised URLs in `node-info.yaml` (the first worker's entry) with the node's own `keytool set-rpc`. It restarts the node once, and only if `node-info.yaml` or the launch command changed; on a committee node it first waits for the epoch boundary (see Restarts and the epoch boundary (page 20)). It records `PUBLIC_RPC_DOMAIN`, `PUBLIC_RPC_URL` and `PUBLIC_WS_URL` in `.node-meta`. Add `--public-ip <ip>` behind NAT. If the dashboard already uses this name, create the A record for `dashboard.<domain>` and add `--move-dashboard-to dashboard.<domain>` to move it in the same run. For a guided run, start `sudo bash ~/telcoin-node-scripts/install-caddy.sh` and choose `[2] Public RPC endpoint`.

The site Caddy serves, block v2 in `/etc/caddy/Caddyfile`, proxies JSON-RPC requests to reth's HTTP port and WebSocket upgrades to its WebSocket port. The upgrade match ignores case, so a proxy that sends `Connection: upgrade` gets through. A browser that opens the URL gets HTTP 405 and a short page naming the endpoint, with a `curl` example. Request bodies above 2 MB (Caddy counts that as 2,000,000 bytes) get HTTP 413, and Caddy cuts the connection once the cap is passed, so reth never receives a whole oversized request. The cap covers JSON-RPC over HTTPS; WebSocket messages are not capped at the edge. Every change backs up the Caddyfile to `/etc/caddy/Caddyfile.bak.<time>`, and the newest five backups are kept.

6.4 Verify

```
curl -s https://$DOMAIN/ -H 'content-type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}'
```

On testnet the answer is `"result":"0x7e1"` (chain ID 2017). The first request after enabling can take a few seconds while the certificate is issued.

For WebSocket, connect with `websocat wss://$DOMAIN/` or `wscat -c wss://$DOMAIN/` and send the same JSON request. Opening `https://$DOMAIN/` in a browser shows the 405 page, which confirms that Caddy serves the name.

```
sudo bash ~/telcoin-node-scripts/check-node.sh
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-status
```

The health check has a public RPC block that probes https and wss through Caddy and shows what `node-info.yaml` advertises. It ends with `public RPC: OK`, or with `public RPC: WARN -- <reasons>` plus the command that fixes it. Run it with `sudo`; without root it can only report `public RPC: unknown (.node-meta not readable — run with sudo)`. `--phase=rpc-status` shows whether the site is enabled, what is advertised, and whether reth's WebSocket port is listening.

A site enabled by an `install-caddy.sh` older than 1.4.0 has the old block: `rpc-status` reports it as stale (`"block_stale": true` with `--json`) and prints the refresh command, and `check-node.sh` warns `public RPC: the Caddy tn-rpc block predates block v2 (...)`. Refresh it by running `rpc-enable` again with the same hostname. That is a Caddy reload; the node restarts only if `node-info.yaml` or its WebSocket flags need a change. The Node Manager UI offers the same refresh as a button.

`check-node.sh` also warns when `--http.api` or `--ws.api` on the node command names `debug`, `trace` or `admin`, because Caddy serves those methods to anyone who reaches the name. Remove them from the launch file and restart the node.

6.5 Turn it off

```
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-disable
```

This removes the Caddy site, clears the advertised URLs from `node-info.yaml` and restarts the node, after the epoch wait on a committee node. It also drops the `PUBLIC_RPC_*` lines from `.node-meta`, and removes the `ufw` rules for 80 and 443 once no Caddy site remains.

7 Validators: stake and activate

The stake and activation calls go to the ConsensusRegistry contract at `0x07E17e17E17e17E17e17E17E17E17e17e17E17e1`, the same address on testnet and devnet. The BLS keys never leave the node. On the node, `prepare-stake.sh` checks everything the stake depends on and prints the exact `cast send` commands; you run those commands on the machine that holds your wallet. The script never reads, asks for or prints a private key, and it sends nothing. The commands in this chapter match the deployed contracts.

The `cast` commands from Watch your status (page 32) on use these variables, set on the wallet machine:

```
REG=0x07E17e17E17e17E17e17E17E17E17e17e17E17e1
RPC=https://rpc.adiri.tel   # testnet; see Networks and endpoints
ADDR=0xYOUR_EXECUTION_ADDRESS
```

7.1 Prerequisites

- Association approval. Only GSMA-approved mobile network operators can validate; start with devs@telcoin.org.
- Hardware at the validator tier (see Hardware per role (page 7)).
- The node's P2P ports reachable from the internet (see Ports (page 8)).
- A synced node (see Confirm the node is syncing, then synced (page 16)).
- A publicly routable RPC URL advertised in `node-info.yaml` (see Public RPC (https and wss) (page 25)). Other nodes forward user transactions to the committee's advertised URLs and skip private addresses. Only a private network, such as a devnet, can lift that with `allow_private_forward_targets: true` in `parameters.yaml` (see Change the configuration (page 21)); `edit-config.sh` refuses `true` on testnet and mainnet.
- The execution address funded with the stake plus gas. The stake is 1,000,000 TEL on testnet today; `prepare-stake.sh` reads the current amount.
- Foundry's `cast` on the wallet machine, and the wallet holding your execution address.

7.2 Check the node with prepare-stake.sh

On the node:

```
sudo bash ~/telcoin-node-scripts/prepare-stake.sh
```

It prints each check as it runs it:

1. Network RPC. The RPC must serve this node's chain: testnet (2017) unless `NETWORK` in `.node-meta` names another network. The default is that network's public RPC (see Networks and endpoints (page 9)); `--network-rpc <URL>` picks another.
2. Validator address. The `execution_address` in `node-info.yaml`, shown with its EIP-55 checksum. It warns when `.node-meta` records a different `VALIDATOR_ADDRESS`.

3. Whitelist. Whether the address holds a ConsensusNFT. Without one the run stops: send the address to the Association for approval, and run the script again once the NFT is minted.
4. Validator status. An address that has not staked goes on. A staked one (status 1) skips to the `activate()` step. Any later status is reported, and the run ends with "Nothing to do".
5. Stake amount. What the registry asks for now, from `stakeConfig(getCurrentStakeVersion())`, in TEL and wei.
6. Balance. The address's TEL against the stake plus a gas allowance (the gas price times a million gas). If it is short, the script says how much to send and still prints the commands.
7. Stake calldata. Exported by the node's own keytool. No passphrase is needed.
8. Simulation. `stake()` is simulated from the address with `eth_estimateGas`. A predicted revert is named with what to do (see If prepare-stake.sh predicts a revert (page 31)), and no commands are printed.
9. Local node sync. A warning when the node trails the network by more than 50 blocks.

Then it prints the stake and activation commands, a note on signing, and the epoch arithmetic from When you get a committee seat (page 33). With `--json` the human output goes to stderr and one JSON object with every value the run read goes to stdout at the end.

The run ends with one line and an exit code:

Exit	Last line	Next step
0	`Ready to stake: run the commands above.`	Send the stake (page 30).
0	`Staked: activate() is next.`	Activate (page 30) once the node has caught up.
0	`Nothing to do: ...`, because activation is under way, the validator is active, exiting or exited, or the address is retired	Watch your status (page 32).
0	`Rotated: node-info.yaml now names 0x....`	Run `prepare-stake.sh` again to export the calldata for the new address.
1	A usage or setup problem, such as `Run this as root: sudo bash prepare-stake.sh <args>`	Do what the message says.
2	The network RPC cannot be reached, serves another chain, or the on-chain state cannot be read; or keytool cannot export the stake calldata	For the RPC, check the server's internet access, or pass `--network-rpc <URL>` with an RPC for the node's chain. For keytool, act on its message, which the last line includes.
3	`Not ready: ...`, `Refused: ...` or `Cancelled: nothing changed.`	Fix what the lines above it name, then run it again.
4	`... node-info.yaml is back as it was (from <backup>).`	A rotation failed and was undone (see Change the execution address before you stake (page 32)).

7.3 Send the stake

The script prints the stake command with your values filled in, for example:

```
cast send --rpc-url https://rpc.adiri.tel 0x07E17e17E17e17E17e17E17E17E17e17e17E17e1 \
  0x2fb0d025... \
  --value 100000000000000000000000 --from 0xYOUR_EXECUTION_ADDRESS --ledger
```

Copy it to the wallet machine and run it there. The calldata holds only public data: the node's BLS public key and its proof of possession.

`--ledger` signs on a Ledger; use `--trezor` for a Trezor. For a key held in software, use `--account <name>`, an encrypted Foundry keystore you create once with `cast wallet import <name> --interactive`, or `--interactive` to paste the key when `cast` asks for it. Never put the key itself on a command line: typed there it lands in your shell history, and passed with `--private-key` it shows in the process list to any user of that machine while `cast` runs. Prefer a hardware wallet for a funded validator address.

Wait for the transaction to be mined. Running `prepare-stake.sh` again then ends with `Staked: activate() is next.` and prints only the activation command.

7.4 Activate

Once the stake has landed (status 1, Staked) and the node has caught up, run the second command the script printed:

```
cast send --rpc-url https://rpc.adiri.tel 0x07E17e17E17e17E17e17E17E17E17e17e17E17e1 \
  'activate()' --from 0xYOUR_EXECUTION_ADDRESS --ledger
```

Status moves to `2` (PendingActivation) and then to `3` (Active) at the next epoch boundary. When you get a committee seat (page 33) says when the first seat can come.

7.5 If prepare-stake.sh predicts a revert

The simulation names the error the registry would return, and the run stops with exit 3 and `Not ready: stake() would revert with <Error>. Fix that before you send the stake.` When a `cast send` reverts, `cast` prints the error selector, and the error's name when it can decode it; the names are the ones below.

Error	What it means	What to do
`InvalidProofOfPossession`	The proof of possession in `node-info.yaml` was signed for a different address than the one you stake from. The keys are fine.	Re-sign it for the staking address with `prepare-stake.sh --rotate-address` (see Change the execution address before you stake (page 32)).
`InvalidTokenId`	The address holds no ConsensusNFT, so it is not whitelisted.	Ask the Association for approval, then run the script again.
`RequiresConsensusNFT`	The ConsensusNFT for the address is not held by that address.	Email devs@telcoin.org.
`InvalidStakeAmount`	The stake configuration changed after the script read it.	Run the script again for the current amount.
`InvalidStatus`	The registry already holds a validator record for the address (the status number is shown), so it cannot stake again.	Check it with `sudo bash ~/telcoin-node-scripts/check-node.sh`.
`DuplicateBLSPubkey`	This BLS key is already registered. A BLS key can be staked only once.	If you staked it from another address, that address is this node's validator: check it with `check-node.sh --address <that address>`. Do not generate new keys.
`InvalidBLSPubkey`	The BLS public key in `node-info.yaml` is not a valid key, so the file was edited or damaged.	Restore `node-info.yaml` from your backup (see Back up the keys now (page 13)).
`EnforcedPause`	The registry is paused, so staking is closed for now.	Try again later.
`LowLevelCallFailure`	The registry could not run its BLS signature check.	Try again later. If it keeps failing, email devs@telcoin.org.
`Error` with a message	The registry rejected the call with that message.	Act on the message.
`Panic`, or an error the script does not know	An internal error in the registry, or error data the script cannot name.	Email devs@telcoin.org with the data the script prints.

7.6 Change the execution address before you stake

The execution address is signed into the proof of possession in `node-info.yaml`. To stake from a different address than the one you entered at key generation, re-sign the proof of possession for it with the same keys:

```
sudo bash ~/telcoin-node-scripts/prepare-stake.sh --rotate-address 0xNEW_ADDRESS
```

Rotation works only while the current address has not staked and is not retired. The new address must not have staked or be retired either; a whitelisted address that has not staked, or one that is not whitelisted yet, is fine. Once an address has staked, the registry holds the BLS key for it and the script refuses with `Refused: 0x... has staked (...)`.

The script:

- Shows what will change and asks first. `--yes` skips the question, and a `--json` run needs it.
- Takes the update lock, so it refuses while `update-node.sh` is applying an update: `Refused: an update is in progress (PID N); try again when it has finished.`
- Copies `node-info.yaml` to `node-info.yaml.bak.<UTC time>` beside it.
- Runs `keytool generate pop` from the node's own release, which changes only `execution_address` and `proof_of_possession`, then checks that the BLS key and the node name are unchanged.
- Records the new address as `VALIDATOR_ADDRESS` in `.node-meta`.
- Restarts the node so it loads the new file, after the epoch wait on a committee node. `--no-restart` leaves the restart to you.

If keytool fails or its result is wrong, the script puts `node-info.yaml` back and exits 4; a wrong passphrase is the usual cause.

The passphrase comes from `TN_BLS_PASSPHRASE` first, then from `/etc/telcoin/bls-passphrase` when that is a regular file with mode 600 or stricter (a LoadCredential install has one), then from a prompt on a terminal. A TPM install deletes the plain passphrase file once it has sealed it, so there type the passphrase at the prompt, or pass the variable through `sudo`, which otherwise drops it:

```
read -rs TN_BLS_PASSPHRASE && export TN_BLS_PASSPHRASE
sudo --preserve-env=TN_BLS_PASSPHRASE bash ~/telcoin-node-scripts/prepare-stake.sh \
  --rotate-address 0xNEW_ADDRESS
```

After a rotation, run `sudo bash ~/telcoin-node-scripts/prepare-stake.sh` again; it exports the calldata for the new address.

7.7 Watch your status

```
cast call $REG \
  "getValidator(address)(address,uint32,uint32,uint8,bool,uint8,uint8)" \
  $ADDR --rpc-url $RPC
```

The seven lines are the validator address, activation epoch, exit epoch, status, `isRetired`, stake version and region. The fourth line is the status (see Status values (page 34)). The same information, with the next step spelled out, comes from:

```
sudo bash ~/telcoin-node-scripts/check-node.sh
```

`check-node.sh` finds the address itself, from `.node-meta` or the node; add `--address <0x...>` to check another one. Its epoch section (see The epoch section (page 16)) shows the current epoch and when it ends, whether the address is in the committee for this epoch and the next two, and a staked validator's activation epoch and earliest committee seat.

Other signals:

- The Node Manager UI shows the validator view as soon as the network reports status 1 to 4 for the node's address, even while the node is still syncing. On a synced node its Current Epoch tile counts down to the boundary and shows the activation epoch and the earliest seat.
- `tn_nodeMode` (see Checking by hand (page 17)) changes from `Observer` to `CvvActive` while the node holds a committee seat. A seated node that falls behind, for example during a long restart, reports `CvvInactive` until it has caught up, then logs `attempting to rejoin consensus` and votes again.
- The current epoch and its committee:

```
cast call $REG "getCurrentEpoch()(uint32)" --rpc-url $RPC
cast call $REG \
  "getCurrentEpochInfo()((address[],uint256,uint64,uint32,uint32,uint8))" \
  --rpc-url $RPC
```

The second call's output starts with the list of committee addresses.

7.8 When you get a committee seat

`activate()` mined in epoch E makes the validator active at epoch E+1. The committee for each epoch is fixed two epochs in advance, so the earliest seat is epoch E+3, the activation epoch plus 2. On testnet's six-hour epochs that is 12 to 18 hours after `activate()`, depending on how much of epoch E was left. `prepare-stake.sh` prints the live numbers, for example:

```
  The network is in epoch 581, which ends in about 2h 10m.
  activate() mined now: active at epoch 582, earliest seat at epoch 584.
```

Selection is random only when there are more eligible validators than seats. Between seats an Active validator reports `Observer`, which is normal.

7.9 Status values

Status	Name	What it means	Your next step
(call reverts)	No record	No ConsensusNFT for this address.	Finish Association approval.
0	Undefined	NFT minted, not staked.	Run `prepare-stake.sh`, then stake (see Check the node with prepare-stake.sh (page 28)).
1	Staked	Stake received.	`activate()` once synced (see Activate (page 30)).
2	PendingActivation	Activation queued.	Wait for the next epoch boundary.
3	Active	Eligible for committee seats.	Keep the node healthy.
4	PendingExit	`beginExit()` called.	Wait until the protocol exits you.
5	Exited	Out of the validator set.	`unstake` from the epoch after the exit epoch; `check-node.sh` says when (see Rewards and exit (page 35)).
6	Any	A reserved sentinel. On its own you should never see it.	Email devs@telcoin.org.
6 with `isRetired` true	Retired	`unstake` ran: NFT burned, stake returned.	None. This identity cannot stake again.

`check-node.sh` prints `Status: Retired` for the last row and appends `(Retired)` to any other status that has `isRetired` set. The scripts treat statuses 1 to 4 as staked and everything else as not staked.

7.10 Stake by hand

`prepare-stake.sh` runs these steps for you. Use them to cross-check its output, or when you cannot run it.

Confirm the ConsensusNFT on the wallet machine; `1` means the Association has minted it, `0` means approval is not complete:

```
cast call $REG "balanceOf(address)(uint256)" $ADDR --rpc-url $RPC
```

Export the stake calldata on the node. The keytool reads only `node-info.yaml`, but it still wants a passphrase source, so pass `--bls-passphrase-source no-passphrase`; `-q` keeps its log line out of the output, so you get the calldata alone. Source builds (an `existing` install uses its own binary path):

```
sudo /opt/telcoin/telcoin-network -q --bls-passphrase-source no-passphrase \
  keytool export-staking-args --node-info /var/lib/telcoin/node-info.yaml --calldata
```

Docker installs:

```
IMAGE=$(sudo sed -n 's/^DOCKER_IMAGE=//p' /etc/telcoin/.node-meta)
sudo docker run --rm -v /var/lib/telcoin:/home/nonroot/data:ro "$IMAGE" \
  telcoin -q --bls-passphrase-source no-passphrase \
  keytool export-staking-args --node-info /home/nonroot/data/node-info.yaml --calldata
```

Copy the `0x...` output to the wallet machine and check that it is calldata for `stake`:

```
CALLDATA=0x...   # paste the output here
[ "${CALLDATA:0:10}" = "$(cast sig 'stake(bytes,(bytes))')" ] && echo "calldata OK"
```

Read the stake amount for the current stake version:

```
VERSION=$(cast call $REG "getCurrentStakeVersion()(uint8)" --rpc-url $RPC)
cast call $REG "stakeConfig(uint8)(uint256,uint256,uint256,uint32)" $VERSION \
  --rpc-url $RPC
STAKE=$(cast call $REG "stakeConfig(uint8)(uint256,uint256,uint256,uint32)" \
  $VERSION --rpc-url $RPC | awk 'NR==1 {print $1}')
```

The four values are the stake amount in wei, the minimum reward withdrawal, the issuance per epoch, and the epoch length in seconds. `stake()` takes exactly the first value of the current epoch's stake version, so send that amount or the call reverts with `InvalidStakeAmount`. Do not read the amount from `getCurrentStakeConfig()`: it returns the newest configuration, which takes effect only at the next epoch. Read the amount just before you stake. Once you have staked, the amount that applies to your validator is fixed by the stake version recorded on it (the sixth field of `getValidator`).

Then stake, and once the stake has landed and the node is synced, activate (Send the stake (page 30) covers the signing options):

```
cast send $REG $CALLDATA --value $STAKE --from $ADDR --ledger --rpc-url $RPC
cast send $REG "activate()" --from $ADDR --ledger --rpc-url $RPC
```

7.11 Rewards and exit

Claim accrued rewards without exiting once they reach `minWithdrawAmount`, the second value of the stake configuration:

```
cast send $REG "claimStakeRewards(address)" $ADDR --from $ADDR --ledger --rpc-url $RPC
```

To leave, call `beginExit()` while Active:

```
cast send $REG "beginExit()" --from $ADDR --ledger --rpc-url $RPC
```

It reverts if too few validators would remain. The status moves to PendingExit (4), and the protocol exits you once no current or upcoming committee needs you. When the status reads Exited (5), wait until the epoch after the exit epoch (`check-node.sh` says when `unstake()` becomes eligible), then reclaim the stake:

```
cast send $REG "unstake(address,bool)" $ADDR false --from $ADDR --ledger --rpc-url $RPC
```

WARNING

`unstake` permanently retires the validator and burns its ConsensusNFT. Neither the address nor the BLS key can stake again, and validating again needs new keys and a new NFT.

The second argument, `acceptRewardShortfall`, stays `false`; `true` forfeits any rewards the Issuance contract cannot cover. `unstake` also works while the status is still Staked (1), before activation.

7.12 What not to do

- Never regenerate keys for a staked identity, and never pass `--force` to the keytool for one. The registered BLS key would no longer match the node, and the registry never releases a registered key, not even after `unstake`.
- Do not generate new keys to change the execution address. Before you stake, re-sign with `prepare-stake.sh --rotate-address` (see Change the execution address before you stake (page 32)); after you stake, the address cannot change.
- New validator keys are a new validator identity. Generate them in a new data directory, and only after talking to the Association.
- Never run the same keys on two servers at once.
- When you re-run `setup-node.sh` on a staked node, answer No to `Overwrite existing keys?`.

8 Automation

8.1 Setup without prompts

`setup-node.sh --json` runs in two phases so you can back up the keys in between. Pass the same flags to both phases. Keygen records the install method, the Docker image or binary path and the node flags in `.node-meta`, and finalize reads them back for any of those you leave out:

```
read -rs TN_BLS_PASSPHRASE && export TN_BLS_PASSPHRASE
IMAGE=us-docker.pkg.dev/telcoin-network/tn-public/adiri:v0.16.0-adiri   # your tag
ARGS=(--network testnet --install-method docker --docker-image "$IMAGE"
  --address 0xYOUR_EXECUTION_ADDRESS
  --external-primary /ip4/203.0.113.10/udp/49590/quic-v1
  --external-worker  /ip4/203.0.113.10/udp/49594/quic-v1
  --listener-primary /ip4/10.0.0.5/udp/49590/quic-v1
  --listener-worker  /ip4/10.0.0.5/udp/49594/quic-v1)
sudo --preserve-env=TN_BLS_PASSPHRASE bash ~/telcoin-node-scripts/setup-node.sh \
  --json --phase=keygen "${ARGS[@]}"
```

Back up `node-keys/` and `node-info.yaml` now (see Back up the keys now (page 13)), then run the second phase in the same shell:

```
sudo bash ~/telcoin-node-scripts/setup-node.sh \
  --json --phase=finalize "${ARGS[@]}"
```

- `keygen` runs preflight, installs dependencies, creates the service user and generates the keys. It writes no unit and starts nothing, and it refuses to run if `node-keys/` already exists.
- `finalize` writes the configuration, creates and starts the service, and enables public RPC when you pass `--rpc-domain`.
- The passphrase comes only from `TN_BLS_PASSPHRASE`; keep it off the command line.
- Each phase prints one JSON object per line on stdout, with `event` set to `step`, `log`, `error` or `done`. Every run ends with exactly one `{"event":"done","ok":true|false,...}`, and a failed run's `done` carries the reason in `msg`. Human-readable output goes to stderr.
- Finalize's `done` adds `operator_guide` (the URL of the operator runbook) and `public_rpc` (`enabled`, `domain`, `http`, `ws`).
- The inputs are checked before anything runs as root. A flag with no value, an unknown phase, network or install method, a malformed URL or image reference, a relative `--binary-path`, and on testnet a `--build-ref` or image tag older than `v0.13.0-adiri` each stop the run with an `error` event. A keygen run also stops there when `--address`, one of the four multiaddr flags or `TN_BLS_PASSPHRASE` is missing. `main`, a commit or an image digest is allowed with a warning.
- `--network` takes `testnet` (or `adiri`) or `devnet`. Mainnet is coming.
- `--install-method` takes `source` (with `--build-ref <tag>`), `docker` or `existing`. Without `--docker-image <ref>`, a Docker keygen picks the newest `-adiri` image and records it.

- For `existing`, `--binary-path <path>` (absolute) names the binary and, in `--json` mode, implies `existing`. Without it, keygen searches PATH, `/usr/local/bin`, `/opt` and `/home`, and stops when it finds none.
- Node flags: `--bootstrap-peers <file>`, `--enable-state-export` and `--state-export-keep <n>` (see Optional node flags (page 14)).
- Other flags: `--data-dir`, `--passphrase-method loadcredential|tpm`, `--public-ip`, `--advertised-name`, `--service-user`, `--service-group`, `--genesis-dir`, `--enable-healthcheck-monitor`, and the public RPC flags from Public RPC (https and wss) (page 25).

8.2 Advertised RPC URL flags

`--rpc-domain <d>` fills in all four of these, so most installs never pass them:

Flag	Written to	Derived from `--rpc-domain <d>`
`--rpc-http <url>`	`node-info.yaml`, published to the network	`https://<d>/`
`--rpc-ws <url>`	`node-info.yaml`; needs `--rpc-http` or `--rpc-domain`, otherwise it is ignored with a warning	`wss://<d>/`
`--public-rpc-url <url>`	`.node-meta PUBLIC_RPC_URL`, for the UI and tooling only	`https://<d>`
`--public-ws-url <url>`	`.node-meta PUBLIC_WS_URL`	`wss://<d>`

An explicit flag wins and is used as given. `--rpc-http` and `--public-rpc-url` take an `http://` or `https://` URL, `--rpc-ws` and `--public-ws-url` a `ws://` or `wss://` one. Setup stops on a malformed URL, and warns when a URL or `--rpc-domain` points at a private address or name (loopback, RFC 1918, `.local`, `.internal`) that wallets on the internet cannot reach. An invalid `--public-ip` is ignored with a warning. Derived URLs reach key generation only when the keytool supports `--rpc-http`; otherwise they are written when `rpc-enable` runs. If an explicit URL differs from the domain, setup warns that `rpc-enable` will replace it.

8.3 Other scripts

Script	Non-interactive forms
`install-caddy.sh`	`--phase=rpc-status`, `rpc-check-dns`, `rpc-enable`, `rpc-disable` and the dashboard phases `status`, `check-dns`, `enable`, `disable`, with or without `--json`. The dashboard password comes from `TN_CADDY_PASSWORD`.
`update-node.sh`	`--json --check`, `--json --prepare --ref <ref>`, `--json --apply --yes [--no-epoch-wait]`, `--json --discard`. A `--ref` value cannot start with `-`. A failed apply rolls back; its `done` line carries `"ok":false` and a boolean `rolled_back` (`true` when the rollback succeeded). A one-way apply (see One-way updates (page 19)) never rolls back: its failed `done` carries `"rolled_back":false` and `"storage_migration":true`, and `--json --check` adds the same boolean, `true` when updating to `latest_ref` is one-way.
`firewall-setup.sh`	`--reset`, `--json --status`, `--json --enable`, `--json --port <49590/udp|49594/udp|43174/tcp> <on|off>`. `--json --status` adds `p2p_ports`, one object per P2P port with `port`, `proto`, `label` and `allowed` (`null` while `ufw` is off).
`edit-config.sh`	`--set <field>=<value>`, with or without `--json`, for the fields in Change the configuration (page 21); `--no-epoch-wait`.
`prepare-stake.sh`	`--json`, `--network-rpc <URL>`, and `--rotate-address <0xNEW> --yes [--no-restart]`. With `--json` it prints one JSON object at the end; the exit codes are in Check the node with prepare-stake.sh (page 28).
`remove-node.sh`	`--json --remove observer|validator --scope service|data|keys --yes [--remove-ui]`. The `--remove` value is a UI slot name, not a role; either one removes the node.

The status and DNS-check phases and `update-node.sh --json --check` print a single JSON object instead of an event stream. In `--json` runs, the epoch wait (see Restarts and the epoch boundary (page 20)) reports as `step`, `log` and `warn` events. `TN_ASSUME_YES=true` answers yes to the scripts' yes/no prompts. `check-node.sh` has no JSON mode and always exits 0, so parse its verdict line.

9 Troubleshooting

Start with `sudo bash ~/telcoin-node-scripts/check-node.sh` and `journalctl -u telcoin -n 200`. The tables below group the common symptoms by area.

9.1 Sync and node mode

Symptom	Likely cause	Fix
`tn_nodeMode` returns `Observer` while `getValidator` shows Active	Not seated yet, or between seats.	Normal. See When you get a committee seat (page 33).
`tn_nodeMode` stays at `CvvInactive`	A committee member that has not caught up. It votes again once it is within 20 rounds of the network's newest commit in the same open epoch, while the network is still committing; the log then shows `attempting to rejoin consensus.`	Treat it like a lagging node (next row).
`CATCHING UP` and the lag does not shrink between runs	Peers cannot reach the node, the disk is full or slow, or the node keeps restarting.	Check the node's P2P ports (see Ports (page 8)) in every firewall layer, check that the external multiaddrs in `node-info.yaml` carry the current public IP, check disk space, and look for restarts in the journal.
`eth_syncing` returns `false` while the node is behind	Expected on Telcoin Network.	Compare `eth_blockNumber` instead (see Checking by hand (page 17)).
`check-node.sh` warns `In the committee for epoch <E>, but the node reports Observer: it is not taking part in consensus`	The node holds a seat in this epoch's committee but is not voting, usually because it is behind or cut off from its peers. If the report adds `The node is at epoch <N>: let it catch up, and it joins once it reaches epoch <E>.`, it is still catching up.	Treat it like a lagging node (the `CATCHING UP` row).
`check-node.sh` reports `In the committee for epoch <E> but NOT in recent headers -- node may be running but silent`	A committee member whose headers the network does not see.	Check that the node runs, that its P2P ports are open in every firewall layer, that its external multiaddrs carry the current public IP, and that time sync is on (see Hardware per role (page 7)).
`check-node.sh` reports `Workers: the network requires <N> per node (...) but node-info.yaml lists <M>`	The node has fewer workers than `WorkerConfigs.numWorkers()` requires.	The scripts cannot add a worker. Email devs@telcoin.org with the `check-node.sh` output.
`eth_sendRawTransaction` returns error `-32000 worker is inactive`	The network's worker count per node went down while this node was running. The node keeps the dropped worker's RPC port open for reads but refuses transactions there.	Send transactions to the first worker's RPC, port 8545 on a default install (see Ports (page 8)).
`check-node.sh` warns `<rpc> serves chain <X> (...), not chain <Y> (...) -- skipping the network comparison`	The comparison RPC serves another chain than the node's. Today `https://rpc.telcoin.network` serves the testnet, because mainnet has not launched.	Not a node problem. Pass `--network-rpc <URL>` with an RPC for the node's chain (see Networks and endpoints (page 9)).

9.2 Setup and preflight

Symptom	Likely cause	Fix
Preflight prints `[WARN] <role>: below minimum`	The server is under that role's minimum.	A warning only. The check counts physical cores, so a cloud instance with hyperthreads has half as many as its vCPUs.
`systemd <version> detected -- version 247+ required.`	The OS is too old.	Move to Ubuntu 22.04+, Debian 12+ or RHEL 9+.
`CVE-2026-31431 (Copy Fail) -- setup cannot continue`	The `algif_aead` module is not blocked, or is still loaded.	Apply the mitigation from https://copy.fail, unload the module or reboot, then re-run setup.
`Port <port>/<proto> (...) is already in use.`	Another process holds the port.	Find it with `sudo ss -lunp` (UDP) or `sudo ss -ltnp` (TCP), then stop it or choose another port.
`--bootstrap-peers needs telcoin-network v0.15.0-adiri or later, and ... does not have it.` (setup), or `the installed release has no --bootstrap-peers; update the node first` (`edit-config.sh`)	The node release lacks the flag. The same messages name `--enable-state-export` (`v0.13.0-adiri`) and `--state-export-keep` (`v0.15.0-adiri`).	Pick a newer release, update the node (see Update the node (page 18)), or leave the flag out.
`<file> is mode 600: other users cannot read it. ... Run chmod 644 <file> first.`	Setup and `edit-config.sh` take only a peers file that other users can read, because the installed copy is world-readable.	`chmod 644 <file>`, then run it again.
`the node rejects <file> as a --bootstrap-peers map (...)`	A key is not a BLS public key, or a value is not the `primary` and `workers` entries of that node's `node-info.yaml`.	Fix the map (see Optional node flags (page 14)).

9.3 Scripts and updates

Symptom	Likely cause	Fix
`lib/common.sh <version> is older than 1.6.0. Run update-scripts.sh and try again.`	The shared library is older than the script that needs it.	`bash ~/telcoin-node-scripts/update-scripts.sh`, then run the script again.
`update-scripts.sh` lists a file as not installed and exits 1	Its published checksum was missing or did not match the download.	Run `update-scripts.sh` again later.
`an update is in progress (PID <N>); try again when it has finished` (`edit-config.sh`), or `Refused: an update is in progress ...` (`prepare-stake.sh --rotate-address`)	`update-node.sh` holds the update lock.	Wait for the update to finish, then run it again.
`Invalid ref "<ref>": a ref cannot start with "-".`	`update-node.sh` refuses a ref that git or docker would read as an option.	Use a release tag, branch, commit or image tag.
`update-node.sh` ends `... Not rolled back: the previous release cannot open the migrated data dir.`	A one-way update to `v0.16.0-adiri` or later did not pass its health check, often because the first start is still migrating the consensus store.	Watch `journalctl -u telcoin -f`. A node that is still migrating comes up without help; a node that stopped says why in the journal. To go back, restore the pre-update snapshot (see One-way updates (page 19)).
Older release fails with `invalid version` (`invalid version (should be 0)` in the log) after a downgrade	`v0.16.0-adiri` or later has migrated the data directory, and older releases cannot read it.	Restore the data directory from the snapshot taken before the update (see One-way updates (page 19)), or run `v0.16.0-adiri` or later again.
`<launch file> has 2 node commands with --http, so it is not clear which one starts the node. ...`	A hand edit left a second node command in the launch file.	Remove or comment out the extra one, then run `edit-config.sh` again.
`firewall-setup.sh` stops right after `Firewall is active` in `1) View current firewall status`	A bug in `firewall-setup.sh` before 1.6.0.	`bash ~/telcoin-node-scripts/update-scripts.sh`.

9.4 Public RPC and certificates

Symptom	Likely cause	Fix
`<domain> already serves the Node Manager dashboard -- the dashboard and the public RPC endpoint need DIFFERENT hostnames`	RPC and dashboard on one name.	Create `dashboard.<domain>` in DNS, then add `--move-dashboard-to dashboard.<domain>` to `rpc-enable`.
`rpc-check-dns` fails, or setup says `Public RPC not enabled yet`	The A record is missing, not propagated, or points at the outbound IP of a NAT host.	Fix the record, pass `--public-ip <inbound-ip>` behind NAT, then re-run `rpc-enable`.
The certificate is not issued, or Let's Encrypt reports a rate limit	DNS or ports 80 and 443 were wrong when Caddy asked.	Fix DNS and the ports first, then wait out the rate limit; repeated attempts extend it.
`port 80 is in use by 'apache2'` (or 443, or nginx)	Another web server holds the ports Caddy needs.	`sudo systemctl disable --now apache2` (or nginx), or run `sudo bash ~/telcoin-node-scripts/install-caddy.sh` and let it guide you.
`Caddy <version> is installed, and this script needs 2.8.0 or newer`	The installed Caddy is too old for the site `install-caddy.sh` writes.	Install the current release from Caddy's apt repository, as the message says, then run it again.
`public RPC: the Caddy tn-rpc block predates block v2 (...)`, or a WebSocket client behind a proxy gets HTTP 405	The site was written by an `install-caddy.sh` older than 1.4.0.	Run `rpc-enable` again with the same hostname (see Verify (page 26)).
`public RPC: --http.api on the node command names <modules>; ...` (or `--ws.api`)	The node serves `debug`, `trace` or `admin` methods, and Caddy passes them to anyone.	Remove them from the launch file and restart the node.
`public RPC: unknown (.node-meta not readable — run with sudo)`	`check-node.sh` ran without root.	Run it with `sudo`.
`wss://<domain>/` returns 502	Nothing listens on the WebSocket port 8546 yet.	Wait if the node is still starting. Otherwise check `install-caddy.sh --phase=rpc-status`, and re-run `rpc-enable` if WebSocket is not listening.

9.5 Service and keys

Symptom	Likely cause	Fix
The node cannot decrypt its BLS key at start	`/etc/telcoin/bls-passphrase` does not hold the passphrase the keys were generated with, or a TPM reset or rebuild stopped a TPM install from unsealing.	Restore the correct passphrase. `edit-config.sh` option 5 rewrites the file but never re-encrypts the keys. A TPM install needs the passphrase you stored offline (TPM notes).
The node will not start; the log shows `another telcoin process (pid <N>) holds the lock on this data directory`	`v0.16.0-adiri` and later lock `<data directory>/telcoin.pid`, and another node process, such as one started by hand, runs on the same data directory.	Stop the other process, then start the service. Do not delete `telcoin.pid`: the lock is released when its holder exits.
The node stops at every start, and the journal shows `Cannot check database (epoch): epoch table <table> holds <n> row(s) this binary cannot decode, the first at key <key> with header epoch <N> ...`	Each start checks the epoch database, `consensus-db/epoch`, which holds only the epoch in progress. It holds a row this release cannot read and every retry finds the same row, so systemd restarts the node every 10 seconds and it can end in `start-limit-hit`.	Email devs@telcoin.org with the journal line first. Then stop the node (`sudo systemctl stop telcoin`) and move the epoch database out of the data directory, for example `sudo mv /var/lib/telcoin/consensus-db/epoch /var/lib/telcoin.epoch-<N>`, and keep `consensus-db/epochs` and `db` in place. That directory also held the node's guards against voting or proposing twice in epoch `<N>`, so start the node again only once the network has closed that epoch: compare `<N>` with the current epoch (see The epoch section (page 16)). A node with state export on can instead be restored into a fresh data directory with `db load-state`.
`systemctl status telcoin` shows `start-limit-hit`	Five failed starts within 60 seconds.	Fix the cause from the journal (for `Cannot check database (epoch)`, see the row above), then `sudo systemctl reset-failed telcoin && sudo systemctl start telcoin`.
After a restart near an epoch boundary, the log shows `previous epoch record missing on restart (killed at its boundary); re-deriving it locally`, `epoch record present but the consensus db tables still hold rows of an older epoch (...); clearing them` or `ignoring parent certificates from another epoch`	The node was stopped while the epoch changed, and it repairs its epoch records as it starts.	Normal, no action needed. For planned restarts, keep to the epoch wait (see Restarts and the epoch boundary (page 20)).
An `ERROR` line ending `; initiating node shutdown: <error>`	The node could not write a vote or certificate durably to its epoch database, `consensus-db/epoch`, and stopped its consensus tasks rather than risk signing twice. It keeps running, but it cannot vote again until it is restarted.	Check the data drive: free space with `df -h /var/lib/telcoin` and I/O errors with `sudo dmesg`. Fix the cause, then restart the node at once with `sudo systemctl restart telcoin`; the epoch wait protects nothing while the node cannot vote. If the line comes back, email devs@telcoin.org with it.

9.6 Staking and exit

Symptom	Likely cause	Fix
`prepare-stake.sh` ends `Not ready: 0x... is not whitelisted.`	The address holds no ConsensusNFT yet.	Send the address to the Association (devs@telcoin.org) and run the script again once the NFT is minted.
`prepare-stake.sh` ends `Not ready: 0x... is short of TEL.`	The balance is below the stake plus gas.	Send at least the amount the line above it names, then run it again.
`prepare-stake.sh` stops with `<rpc> serves chain <X> (...), not chain <Y> (...)`	The network RPC serves another chain than the node's.	Pass `--network-rpc <URL>` with an RPC for the node's chain (see Networks and endpoints (page 9)).
`stake` reverts, or `prepare-stake.sh` ends `Not ready: stake() would revert with <Error>`	The registry would refuse the stake.	Look the error up in If prepare-stake.sh predicts a revert (page 31).
`activate` reverts with `InvalidStatus`	The validator is not in the status the call needs.	Check the status (see Watch your status (page 32)).
`check-node.sh` warns `VALIDATOR_ADDRESS in .node-meta (...) is not the node's execution address (...)`	`.node-meta` names another address than `node-info.yaml`, for example after a hand edit. The stake status is read for the `.node-meta` address.	Correct `VALIDATOR_ADDRESS` in `/etc/telcoin/.node-meta`, or pass `--address`.
`No BLS key passphrase. Set TN_BLS_PASSPHRASE ...`	`--rotate-address` found no passphrase source.	Use one of the sources in Change the execution address before you stake (page 32).
`Refused: 0x... has staked (...)`	The address has staked, so its BLS key stays registered to it.	Stake from that address; it cannot change now.
`prepare-stake.sh --rotate-address` exits 4 with `keytool generate pop failed (exit <N>): ... node-info.yaml is back as it was (from <backup>).`	Usually a wrong passphrase.	Run it again with the passphrase the keys were generated with.
`beginExit` reverts	The validator is not Active yet, or too few validators would remain.	Wait, then retry.
`unstake` reverts with `IneligibleUnstake`	The validator is neither Staked nor one epoch past Exited.	Wait for Exited plus one epoch.

10 Getting help

Email devs@telcoin.org for everything, including validator onboarding, approval and hardware.

Include:

- The full output of `sudo bash ~/telcoin-node-scripts/check-node.sh`.
- For staking questions, the output of `sudo bash ~/telcoin-node-scripts/prepare-stake.sh`.
- The last 200 log lines: `journalctl -u telcoin -n 200 --no-pager`.
- The node version (the Docker image tag, or the tag you built) and the script versions that `update-scripts.sh` lists.
- The network (testnet or devnet) and your install method.

> **WARNING**
>
> Never send the contents of `node-keys/` or your BLS passphrase, to support or to anyone else.